

POLYNOMIAL REGRESSION

What Is Regression in Machine Learning?

Regression is a supervised learning technique that models the relationship between input features and a continuous target variable, using statistical methods to predict the target variable based on new input data.

Regression models sift through large numbers of variables, identifying those with the greatest impact outcomes. Regression is foundational to machine learning, especially for predictive use cases. By fitting a regression model to data, organizations can replace educated guesses and hunches with data-driven insights into the factors most likely to drive future outcomes and behaviors.

1. Introduction

Polynomial regression is an extension of linear regression that models the relationship between an independent variable (X) and a dependent variable (Y) as an nth-degree polynomial. Unlike simple linear regression, which fits a straight line, polynomial regression can capture non-linear patterns in data.

This project focuses on implementing polynomial regression using Python to analyze datasets where relationships between variables are curved rather than linear.

2. Objective of the Project

- To understand the concept of polynomial regression.
- To implement polynomial regression using Python libraries.
- To visualize how polynomial curves fit complex datasets.
- To evaluate model accuracy using appropriate metrics.

3. Concept of Polynomial Regression

Polynomial regression transforms the original input features into higher-degree terms such as:- $\sqrt{x^2}$, x^3 , x^4 , ...

The general equation of polynomial regression is:

$$[y = b_0 + b_1x + b_2x^2 + b_3x^3 ++ b_nx^n]$$

Where:

- y = dependent variable
- x = independent variable
- b_0, b_1, b_n = coefficients

This allows the model to fit curved relationships in the data.

4. Tools and Technologies Used

- Programming Language: Python
- Libraries:
 - NumPy (numerical operations)
 - Pandas (data handling)
 - Matplotlib / Seaborn (data visualization)
 - Scikit-learn (model building)

5. Methodology

Step 1: Data Collection

A dataset is selected that contains a non-linear relationship between variables (e.g., salary vs experience, temperature vs sales).

Step 2: Data Preprocessing

- Handling missing values
- Splitting dataset into training and testing sets
- Feature scaling (if required)

Step 3: Feature Transformation

Using polynomial features:

```
from sklearn.preprocessing import PolynomialFeatures  
  
poly = PolynomialFeatures(degree=2)  
  
X_poly = poly.fit_transform(X)
```

Step 4: Model Training

```
from sklearn.linear_model import LinearRegression
```

```
model = LinearRegression()
```

```
model.fit(X_poly, y)
```

Step 5: Visualization

Plotting original data points and regression curve to observe fitting.

Step 6: Model Evaluation

- Mean Squared Error (MSE)
- R^2 Score (coefficient of determination)

6. Applications of Polynomial Regression

Polynomial regression is widely used in:

- Predicting economic trends
- Stock market analysis
- Growth curve modeling
- Engineering design problems
- Medical research (dose-response curves)
- Environmental modeling

It is especially useful when the data shows a curved trend instead of a straight-line relationship.

7. Advantages

- Captures non-linear relationships effectively
- Simple to implement using existing linear regression tools
- Flexible model with adjustable degree

8. Limitations

- High risk of overfitting with higher-degree polynomials
- Sensitive to outliers

- Can become computationally expensive for large datasets
- Interpretation becomes difficult as degree increases

9. Challenges Faced During the Project

1. Choosing the Right Degree

One of the biggest challenges was selecting the correct polynomial degree:

- Low degree → underfitting
- High degree → overfitting

Finding the balance required experimentation and validation.

2. Overfitting

Higher-degree models started fitting noise instead of the actual pattern, reducing prediction accuracy on new data.

3. Data Quality Issues

- Missing values affected model training
- Outliers distorted curve fitting

4. Visualization Complexity

Plotting higher-degree curves and interpreting them was sometimes difficult.

5. Computational Complexity

As the degree increased, the number of features also increased, leading to slower computations.

10. Results and Observations

- Polynomial regression provided a significantly better fit compared to linear regression for non-linear data.
- The model's accuracy improved up to a certain degree, after which performance declined due to overfitting.
- Visualization confirmed that polynomial curves captured the trend more accurately.

11. Conclusion

Polynomial regression is a powerful technique for modeling non-linear relationships. Through this project, we learned how transforming features can improve model performance. However, careful selection of polynomial degree and proper validation techniques are crucial to avoid overfitting.

This project demonstrated that while polynomial regression is simple to implement, achieving optimal performance requires thoughtful experimentation and evaluation.

12. Future Scope

- Use cross-validation for better model selection
- Apply regularization techniques (Ridge, Lasso)
- Work with real-world large datasets
- Explore other non-linear models like decision trees and neural networks

```

1  import tkinter as tk
2  from tkinter import filedialog, messagebox, ttk
3
4  import matplotlib
5  import numpy as np
6  import pandas as pd
7  from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg
8  from matplotlib.figure import Figure
9
10 from sklearn.model_selection import train_test_split
11
12 matplotlib.use("TkAgg")
13
14 # MODEL
15 class PolynomialRegression:
16     def __init__(self, degree, lr=1e-1, epoch=10000):
17         self.lr = lr
18         self.epoch = epoch
19         self.degree = degree
20
21     def fit(self, x, yt):
22         if yt.ndim == 1:
23             yt = yt.reshape(-1, 1)
24
25         if x.ndim == 1:
26             x = x.reshape(-1, 1)
27
28         np.random.seed(42)
29         X = [(x**i) for i in range(1, self.degree + 1)]
30         X = np.concatenate(X, axis=1)
31
32         m, n = X.shape
33         self.w = np.random.randn(n, 1)
34         self.b = np.random.randn(1,)
35
36         self.mu = X.mean(axis=0)
37         self.std = X.std(axis=0)

```

```

38         X = (X - self.mu) / self.std
39
40         prev_loss = float('inf')
41
42         for i in range(self.epoch):
43             yp = (X @ self.w + self.b).reshape(-1, 1)
44             loss = 0.5 * ((yp - yt) ** 2).mean()
45
46             grad = (X.T @ (yp - yt)) / m
47             self.w -= self.lr * grad
48             self.b -= self.lr * (yp - yt).mean()
49
50             if abs(prev_loss - loss) < 1e-6:
51                 break
52             prev_loss = loss
53
54     def predict(self, x):
55         X1 = [(x**i) for i in range(1, self.degree + 1)]
56         X1 = np.concatenate(X1, axis=1)
57         X1 = (X1 - self.mu) / self.std
58         return X1 @ self.w + self.b
59
60
61     def r2_score(yt, yp):
62         sse = ((yt - yp) ** 2).sum()
63         sst = ((yt - yt.mean()) ** 2).sum()
64         return 1 - (sse / sst)
65
66
67     def best_fit(x, y):
68         x_train, x_test, y_train, y_test = train_test_split(
69             x, y, test_size=0.2, random_state=42
70         )
71

```

```

72     x_train, x_test = x_train.to_numpy(), x_test.to_numpy()
73     y_train, y_test = y_train.to_numpy(), y_test.to_numpy()
74
75     y_test = y_test.reshape(-1, 1)
76
77     models = [PolynomialRegression(degree=i) for i in range(1, 5)]
78
79     for model in models:
80         model.fit(x_train, y_train)
81
82     yps = np.array([model.predict(x_test).ravel() for model in models]).T
83
84     sse = ((y_test - yps) ** 2).sum(axis=0)
85     sst = ((y_test - y_test.mean()) ** 2).sum()
86
87     score = 1 - (sse / sst)
88     return np.argmax(score)
89
90
91 def build_regression_model(degree):
92     return PolynomialRegression(degree)
93
94
95 def preprocess(df, x_col, y_col):
96     y = df[y_col]
97     x = df[x_col]
98
99     cat_cols = x.select_dtypes(include=['object']).columns
100    num_cols = x.select_dtypes(include=['int64', 'float64']).columns
101
102    x = pd.get_dummies(x, columns=cat_cols, dtype=int)
103    x[num_cols] = (x[num_cols] - x[num_cols].mean()) / x[num_cols].std()
104
105    return x, y
106

```

```

108 # GUI
109 class RegressionApp:
110     def __init__(self, root):
111         self.root = root
112         self.root.title("Simple Regression Analyzer")
113         self.root.geometry("1100x750")
114
115         self.dataframe = None
116         self.numeric_columns = []
117         self.current_file = None
118
119         self.file_var = tk.StringVar(value="No CSV selected")
120         self.shape_var = tk.StringVar(value="X shape: -   Y shape: -")
121         self.metrics_var = tk.StringVar(value="Load CSV to begin.")
122         self.status_var = tk.StringVar(value="Auto detect enabled")
123
124         self._build_layout()
125
126     def _build_layout(self):
127         outer = ttk.Frame(self.root, padding=15)
128         outer.pack(fill="both", expand=True)
129
130         # ===== TOP =====
131         top = ttk.LabelFrame(outer, text="CSV Input", padding=10)
132         top.pack(fill="x", pady=10)
133
134         ttk.Button(top, text="Open CSV", command=self.open_csv)\
135             .grid(row=0, column=0, padx=5, pady=5)
136
137         ttk.Label(top, textvariable=self.file_var)\
138             .grid(row=0, column=1, columnspan=3, sticky="w")
139
140         ttk.Label(top, text="Target (Y):")\
141             .grid(row=1, column=0, padx=5)
142
143         self.target_combo = ttk.Combobox(top, state="readonly", width=25)
144         self.target_combo.grid(row=1, column=1, columnspan=3, sticky="w")

```

```

143 self.target_combo = ttk.Combobox(top, state="readonly", width=25)
144 self.target_combo.grid(row=1, column=1)
145
146 ttk.Button(top, text="Run Analysis", command=self.run_analysis)\
147 | .grid(row=1, column=2, padx=10)
148
149 ttk.Label(top, textvariable=self.shape_var)\
150 | .grid(row=2, column=0, columnspan=4, sticky="w")
151
152 ttk.Label(top, textvariable=self.status_var, foreground="gray")\
153 | .grid(row=3, column=0, columnspan=4, sticky="w")
154
155 # ===== MIDDLE =====
156 middle = ttk.Frame(outer)
157 middle.pack(fill="both", expand=True)
158
159 left = ttk.LabelFrame(middle, text="Preview", padding=10)
160 left.pack(side="left", fill="both", padx=5)
161
162
163
164 right = ttk.LabelFrame(middle, text="Results", padding=10)
165 right.pack(side="left", fill="both", expand=True)
166
167 self.metrics_label = ttk.Label(right, textvariable=self.metrics_var, justify="left")
168 self.metrics_label.pack(anchor="w", pady=5)
169
170 self.figure = Figure(figsize=(8, 5))
171 self.ax1 = self.figure.add_subplot(211)
172 self.ax2 = self.figure.add_subplot(212)
173
174 self.canvas = FigureCanvasTkAgg(self.figure, master=right)
175 self.canvas.get_tk_widget().pack(fill="both", expand=True)
176

```

```

178 def open_csv(self):
179     path = filedialog.askopenfilename(filetypes=[("CSV", "*.csv")])
180     if not path:
181         return
182
183     df = pd.read_csv(path)
184
185     self.dataframe = df
186     self.numeric_columns = df.select_dtypes(include=[np.number]).columns.tolist()
187
188     self.file_var.set(path)
189     self.target_combo["values"] = self.numeric_columns
190     self.target_combo.set(self.numeric_columns[-1])
191     self.run_analysis()
192
193
194
195 def run_analysis(self):
196     if self.dataframe is None:
197         return
198
199     y_col = self.target_combo.get()
200     x_cols = [c for c in self.numeric_columns if c != y_col]
201
202     df = self.dataframe[x_cols + [y_col]].dropna()
203
204     X, y = preprocess(df, x_cols, y_col)
205
206     score = best_fit(X, y)
207
208     model = build_regression_model(score + 1)
209
210     X_train, X_test, y_train, y_test = train_test_split(
211         X.to_numpy(), y.to_numpy(), test_size=0.2, random_state=42
212     )

```



```

213     model.fit(X_train, y_train)
214     preds = model.predict(X_test)
215
216     y_test = y_test.reshape(-1, 1)
217
218     mse = ((y_test - preds) ** 2).mean()
219     rmse = np.sqrt(mse)
220     mae = np.mean(np.abs(y_test - preds))
221     r2 = r2_score(y_test, preds)
222
223     if score==0:
224         scre='Linear Regression'
225     else:
226         scre=f'Polynomial Regression of Degree {score+1}'
227     self.metrics_var.set(
228         f"Best Model: {scre}\n"
229         f"R2: {r2:.4f}\nMSE: {mse:.4f}\nRMSE: {rmse:.4f}\nMAE: {mae:.4f}"
230     )
231
232     self.ax1.clear()
233     self.ax2.clear()
234     min_val = min(float(np.min(y_test)), np.min(preds))
235     max_val = max(np.max(y_test), np.max(preds))
236
237     self.ax1.scatter(y_test, preds)
238     self.ax1.plot([min_val,max_val],[min_val,max_val],'r--',color='red')
239
240     self.ax1.set_title("Actual vs Predicted")
241
242     residuals = y_test - preds
243     self.ax2.scatter(preds, residuals)
244     self.ax2.axhline(0, color="red", linestyle="--", linewidth=1.5)
245     #self.ax2.axhline(0)

```

```

249
250
251 def main():
252     root = tk.Tk()
253     RegressionApp(root)
254     root.mainloop()
255
256
257 if __name__ == "__main__":
258     main()

```

CSV DATA FILE USED FOR THE OUTPUT SHOWN ON NEXT PAGE...

	A	B	C	D	E	F	
1	Hours Studied	Previous Scores	Extracurricular Activities	Sleep Hours	Sample Question Papers	Performance Index	
2	7	99	Yes	9	1	91	
3	4	82	No	4	2	65	
4	8	51	Yes	7	2	45	
5	5	52	Yes	5	2	36	
6	7	75	No	8	5	66	
7	3	78	No	9	6	61	
8	7	73	Yes	5	6	63	
9	8	45	Yes	4	6	42	
10	5	77	No	8	2	61	
11	4	89	No	4	0	69	
12	8	91	No	4	5	84	
13	8	79	No	6	2	73	
14	3	47	No	9	2	27	
15	6	47	No	4	2	33	
16	5	79	No	7	8	68	
17	2	72	No	4	3	43	
18	8	73	Yes	8	4	67	
19	6	83	Yes	7	2	70	
20	2	54	Yes	4	9	30	
21	5	75	No	7	0	63	
22	1	99	Yes	4	3	71	
23	6	96	No	9	0	85	
24	9	74	Yes	7	6	73	
25	1	85	No	5	6	57	
26	3	61	No	6	3	35	



